

Analysis Real Case

Group : 7 Siang

1. The Specific Problem

Ticketmaster sebenarnya menghadapi masalah saat war ticket acara besar, yaitu lonjakan trafik yang bisa menembus jutaan akses hanya dalam hitungan detik. Kalau semua request read/write ini langsung ditangani oleh database relasional utama mereka, sistemnya pasti akan lelah untuk memproses antrian data atau lock contention yang pada akhirnya berujung pada server crash atau downtime. Selain masalah performa, mereka juga harus berpikir untuk menghindari race condition yaitu insiden tiket terjual melebihi kuota karena dibeli bersamaan oleh banyak orang. Tantangan penting lainnya adalah mengelola sesi pembayaran secara akurat di mana sistem butuh cara untuk lock sementara kursi yang sedang dalam proses bayar, dan otomatis melepaskannya kembali ke pasar jika pembeli tersebut melewati batas waktu yang ditentukan.

2. Alasan Memilih Paradigma Key-Value Store Dibandingkan RDBMS atau NoSQL Lainnya

Untuk mengatasi masalah itu, Ticketmaster akhirnya memilih paradigma Key-Value Store, dengan menjadikan Redis sebagai pemeran utamanya. Alasan dasarnya adalah database relasional konvensional terlalu berat karena berjalan di atas disk dan menggunakan sistem transaksi yang kompleks, sehingga justru akan menciptakan bottleneck saat dipakai oleh banyak pengguna sekaligus. Sebaliknya, Redis bekerja sepenuhnya di dalam RAM atau in-memory yang membuatnya mampu merespons permintaan dengan sangat cepat. Selain itu, Redis memiliki fitur operasi atomik seperti DECR yang menjamin pengurangan stok tiket terjadi secara berurutan dan akurat tanpa takut tiket jadi minus. Fitur yang paling penting di sini adalah Time-To-Live atau TTL. Daripada kita memberatkan server dengan menjalankan perintah rumit untuk menghapus sesi yang expired seperti di database relasional, key atau kursi yang dikunci di Redis akan hancur dengan sendirinya begitu waktu habis. Kita tidak menggunakan NoSQL tipe lain karena tipe Document atau Graph terlalu rumit untuk kebutuhan ini. Key-Value adalah struktur paling sederhana, paling ringan, dan juga paling cepat untuk menangani skenario war ticket.

3. High-Level Data Model & Architecture

Secara garis besar arsitektur, Ticketmaster memosisikan Redis sebagai lapisan pelindung di garis terdepan sistemnya. Semua traffic liar saat war ticket akan ditangani terlebih dahulu oleh Redis menggunakan tipe-tipe data yang sederhana. Misalnya, antrean user ditampung dalam virtual waiting room menggunakan tipe data List yang memproses antrean secara adil dari depan ke belakang atau FIFO. Kuota tiket cukup disimpan sebagai angka String/Integer yang dikurangi di dalam RAM. Sementara itu, untuk menahan kursi pembeli, sistem memakai String yang diberi timer kadaluarsa yaitu TTL. Setelah di Redis ini selesai dan seorang user berhasil menyelesaikan pembayarannya, barulah data transaksi yang sudah benar-benar valid tersebut dikirim ke database relasional utama secara background untuk disimpan secara permanen.

Mini Project

1. Assigned Paradigm & Tool

- Paradigm: Key-Value Store
- Tool/Database: Redis (Remote Dictionary Server)

2. Real Case

Company: Ticketmaster

Specific Use Case: Distributed Ticketing Lock.

Analysis:

Ticketmaster sering menangani penjualan tiket event terkenal yang dapat memicu jutaan akses bersamaan dalam hitungan detik. Jika mereka langsung memproses semua request tersebut ke relational database utamanya, sistem akan mengalami lock contention (antrian baca/tulis yang macet) dan berujung crash. Ticketmaster sebagai solusi yang menempatkan arsitektur Key-Value Store yang berjalan in-memory. Mereka memanfaatkannya untuk sistem Virtual Waiting Room guna mengatur antrian pengguna secara real-time. Selain itu, mereka menggunakan mekanisme Distributed Locking dengan fitur Key Expiry (TTL). Saat seorang user sedang di halaman pembayaran, kursi tersebut di-lock secara atomik sebagai key sementara di Redis. Jika pembayaran selesai, tiket diterbitkan, tetapi jika batas waktu TTL habis sebelum dibayar, key otomatis hancur dan kursi kembali tersedia untuk pembeli lain tanpa membebani database utama sama sekali.

3. Creative Project Idea

Title: TicketFlash: High-Speed Event Ticketing & Flash Sale System

E-Ticketing / Sistem Reservasi Real-Time

4. Problem Statement

Saat ada suatu konser atau acara yang diminati banyak orang, kadang sistem *War Ticket* yang ada kurang memadai untuk jumlah pengguna. Server kadang mengalami crash dan database relasional gagal menangani ribuan permintaan read/write yang dilakukan secara bersamaan. Hal ini sering memicu race condition, dimana tiket terjual melebihi kuota.

Kami menawarkan solusi TicketFlash. Ini adalah sistem booking berkecepatan tinggi yang memanfaatkan arsitektur in-memory Redis untuk mengeksekusi pengurangan stok secara atomik, memegang sesi tiket sementara menggunakan Time-To-Live, dan menyediakan pembaruan sisa kuota secara real-time.

5. Conceptual Data Model

Sistem ini menggunakan struktur data yang disesuaikan dengan kekuatan Key-Value Store (Redis) yang berjalan in-memory. Pemodelan menghindari relasi antar tabel (seperti di SQL) dan berfokus pada kecepatan akses:

- Event Details (Data Type: Hash)
 - Key Format: event:{event_id} (Contoh: event:KMP123)
 - Value: { name: "Konser Kampus", price: 50000, date: "2024-12-01" }
 - Event details menggunakan struktur Hash untuk menyimpan detail statis acara agar bisa diakses dengan latensi O(1) saat halaman web dimuat oleh ribuan pengguna secara bersamaan.
- Stock Counter (Data Type: String / Integer)
 - Key Format: ticket_stock:{event_id} (Contoh: ticket_stock:KMP123)
 - Value: 500 (integer)
 - Stock Counter berfungsi untuk menyimpan total tiket yang tersedia. Saat war ticket terjadi, sistem akan menggunakan operasi DECR (Decrement) bawaan Redis yang bersifat atomik. Ini menjamin tidak akan terjadi race condition seperti tiket yang terhitung ganda atau minus meskipun ribuan orang menekan "Beli" di waktu yang sama.
- Distributed Lock & Temporary Hold (Data Type: String dengan TTL)
 - Key Format: hold:{event_id}:{user_id} (Contoh: hold:KMP123:USR99)
 - Value: "locked"
 - TTL (Time-To-Live): 300 seconds (5 Menit)
 - Distributed Lock & Temporary Hold bertindak saat user berhasil mendapat kuota, key ini dibuat dengan waktu kadaluarsa (TTL) 5 menit untuk proses pembayaran. Jika user tidak membayar dalam 5 menit, key otomatis hancur (hilang dari Redis). Aplikasi akan mendeteksi hilangnya key ini dan mengembalikan stok tiket (+1) secara otomatis.
- Virtual Waiting Room (Data Type: List)
 - Key Format: queue:{event_id} (Contoh: queue:KMP123)
 - Value: ["USR99", "USR105", "USR22..."]
 - Jika trafik melebihi batas, pengguna baru akan dimasukkan ke dalam antrian menggunakan operasi R PUSH (masuk antrian belakang) dan akan diproses secara FIFO (First-In-First-Out) dengan LPOP (keluar antrian depan). Menjamin keadilan bagi peserta war ticket.

6. Role Distribution

- **Arya Wibawa Atmanegara (2406420431):** Conceptual Data Model, The Specific Problem
- **Clementheo Benaya Raya (2406413810):** Problem Statement, Creative Project Idea, Alasan Memilih Paradigma Key-Value Store
- **Haidar Rafif Radithya (2406408836):** Real Case, The Specific Problem, Creative Project Idea
- **Sutan Rendy Rizaldi (2406434626):** Assigned Paradigm & Tool, Alasan Memilih Paradigma Key-Value Store
- **Yusri Sukur (2406345305):** Conceptual Data Model, High-Level Data Model & Architecture